

Synkora Studio

AI-Native Product Specification Platform

Product positioning: Figma for Product Specs

Implementation model: Vertical application built inside the Synkora codebase

Primary deployment: Deriv-controlled infrastructure

Document type: Product requirements, system design, and implementation specification

Intended implementers: Codex, Claude Code, software engineers, product engineers and platform engineers

Status: Build-ready specification

Priority: MVP first, extensible architecture throughout

1. Product Definition

Synkora Studio is a web-first, collaborative platform that transforms a product idea into a structured, connected and reviewable product specification.

A user enters an idea such as:

Build an AI-powered CRM for financial institutions that helps relationship managers prepare for meetings, understand customers and manage opportunities.

Studio generates and connects:

```
graph TD
    Idea --> ProblemStatement[Problem Statement]
    ProblemStatement --> ResearchFindings[Research Findings]
    ResearchFindings --> Personas[Personas]
    Personas --> BusinessObjectives[Business Objectives]
    BusinessObjectives --> ProductRequirements[Product Requirements]
    ProductRequirements --> Features[Features]
    Features --> UserStories[User Stories]
    UserStories --> AcceptanceCriteria[Acceptance Criteria]
    AcceptanceCriteria --> UserExperience[User Experience]
```

```
Architecture
↓
Data Model
↓
APIs
↓
Events
↓
Security Controls
↓
Delivery Plan
↓
Tests
↓
Deployment
↓
Monitoring
↓
Product Outcomes
```

The defining feature is not document generation.

The defining feature is that every artefact remains structured, versioned and connected.

For example:

```
Business Objective
├─ supported by → Feature
│   ├── defined by → Requirement
│   ├── delivered through → User Story
│   ├── exposed through → API Operation
│   ├── implemented by → Backend Service
│   ├── stores data in → Data Entity
│   ├── verified by → Test Case
│   ├── deployed through → Deployment
│   └─ measured by → Product Metric
```

Studio must make these relationships visible through a canvas and usable through conventional document views.

2. Product Vision

2.1 Vision

Create a single collaborative workspace where a team can move from product intent to engineering delivery while preserving traceability between decisions, requirements, implementation, testing, deployment and production outcomes.

2.2 Product promise

Describe the product once. Develop, review and maintain its complete specification from one connected workspace.

2.3 Product analogy

Figma:
Source of truth for collaborative interface design

GitHub:
Source of truth for code

Synkora Studio:
Source of truth for product intent and implementation context

2.4 Relationship with Synkora

Synkora remains the reusable AI application platform.

Synkora Studio becomes a vertical application powered by Synkora.

Synkora Core

- ├ Authentication
- ├ Workspaces
- ├ Agent runtime
- ├ Model gateway
- ├ Knowledge bases
- ├ Retrieval
- ├ Tools
- ├ Workflows
- ├ Integrations
- ├ Human approvals
- ├ Background processing
- └ Observability

Synkora Studio

- ├ Products

- └─ Product artefacts
- └─ Artefact graph
- └─ Specification canvas
- └─ Document projections
- └─ Reviews
- └─ Versioning
- └─ Quality checks
- └─ Product agents
- └─ Delivery traceability

Studio must not replace or duplicate Synkora Core services.

3. Core Product Principles

3.1 The graph is the source of truth

Documents are generated views of structured artefacts.

A PRD is not stored as one large Markdown document. It is rendered from product objectives, personas, requirements, features, risks and metrics.

3.2 AI proposes; people control

AI may create drafts, suggestions, relationships, analyses and quality findings.

AI must not silently overwrite approved content.

3.3 Web-first experience

All primary workflows must be available through a polished graphical interface.

CLI access may exist for automation but must not be required for normal usage.

3.4 Progressive generation

The system must generate products in stages.

Generate → Review → Approve → Continue

It must not run one large, uncontrolled prompt that generates the entire product.

3.5 Structured output

All AI-generated artefacts must conform to versioned JSON schemas.

3.6 Traceability by default

Every feature should be traceable from business objective to production metric.

3.7 Studio is modular

Studio-specific logic must remain within Studio modules.

Synkora Core must remain usable without Studio.

3.8 Integrate rather than replace

Studio should link and synchronise with GitHub, project-management tools, Figma, Slack, CI/CD and observability platforms.

It should not attempt to replace every external system.

4. Product Goals

4.1 Primary goals

1. Transform natural-language ideas into structured product specifications.
2. Maintain typed relationships between product, design, technical and operational artefacts.
3. Provide a visual canvas for exploring and editing product specifications.
4. Generate conventional documents from the underlying graph.
5. Support collaborative review, comments, approvals and version history.
6. Provide specialist AI agents for each product-development discipline.
7. Detect missing, conflicting or stale specification content.
8. Generate an engineering-ready handoff.
9. Integrate specifications with implementation systems.
10. Run within Deriv-controlled infrastructure.

4.2 Secondary goals

1. Standardise specification quality across teams.
2. Reduce PRD and technical-design preparation time.
3. Introduce security and observability requirements earlier.
4. Reduce repeated context preparation for coding agents.
5. Improve onboarding into existing products.
6. Make product decisions discoverable and auditable.
7. Provide consistent context to Codex, Claude Code and other coding agents.

4.3 MVP non-goals

The first release will not:

- Replace GitHub.

- Replace Figma.
 - Replace Jira, Linear or ClickUp.
 - Generate and deploy an entire production application autonomously.
 - Automatically approve specifications.
 - Automatically deploy production changes.
 - Provide a complete whiteboarding product.
 - Provide portfolio financial planning.
 - Support complex real-time multiplayer text editing.
 - Use a dedicated graph database.
 - Implement every artefact in the full product lifecycle.
 - Synchronise deeply with every external tool.
-

5. Target Users

5.1 Product manager

Primary workflows:

- Create a product from an idea.
- Generate and refine a PRD.
- Define objectives and metrics.
- Review requirements, features and stories.
- Request stakeholder approval.
- Track specification readiness.
- Generate engineering handoff material.

5.2 Engineering lead

Primary workflows:

- Review product requirements.
- Generate or edit architecture.
- Identify impacted components.
- Review APIs and data models.
- Identify technical risks.
- Approve engineering readiness.

5.3 Software engineer

Primary workflows:

- Understand why a task exists.
- View connected requirements and design context.
- Generate an implementation plan.
- Export structured context to a coding agent.
- Link implementation work to the specification.
- Identify spec changes required by implementation.

5.4 Designer

Primary workflows:

- Review personas and journeys.
- Define user flows.
- Link Figma designs.
- Check screens against requirements.
- Identify missing states.

5.5 QA engineer

Primary workflows:

- Generate test scenarios.
- Map acceptance criteria to test cases.
- Detect coverage gaps.
- Link automated test results.
- Review regression impact.

5.6 Security reviewer

Primary workflows:

- Review data classifications.
- Generate threat models.
- Add security requirements.
- Identify missing controls.
- Approve high-risk artefacts.

5.7 Stakeholder

Primary workflows:

- Review product goals and scope.
- Comment on artefacts.
- Approve assigned decisions.
- View progress, risks and expected outcomes.

5.8 Workspace administrator

Primary workflows:

- Configure Studio availability.
- Manage roles and access.
- Configure product templates.
- Configure AI models.
- Configure agents and integrations.
- Review audit logs and usage.

6. Product Scope

6.1 Full target lifecycle

```
Idea
Discovery
Product Definition
Experience Design
Technical Design
Security and Compliance
Delivery Planning
Implementation
Testing
Deployment
Operations
Outcome Review
```

6.2 MVP lifecycle

The MVP must implement this vertical slice:

```
Idea
↓
Problem Statement
↓
Business Objectives
↓
Personas
↓
Requirements
↓
Features
↓
User Stories
↓
Acceptance Criteria
↓
High-Level Architecture
↓
API Outline
↓
Data Model
↓
Test Strategy
↓
Engineering Handoff
```

7. Architecture Strategy

7.1 Implementation rule

Build Studio inside the existing Synkora monorepo and reuse the current architecture.

Do not create a permanently divergent repository.

Recommended logical structure:

```
synkora-ai/
├── api/
│   └── src/
│       ├── studio/
│       │   ├── api/
│       │   ├── models/
│       │   ├── schemas/
│       │   ├── repositories/
│       │   ├── services/
│       │   ├── agents/
│       │   ├── workflows/
│       │   ├── quality/
│       │   ├── integrations/
│       │   └── tasks/
│       └── existing-synkora-modules/
├── web/
│   ├── app/
│   │   └── studio/
│   ├── components/
│   │   └── studio/
│   ├── hooks/
│   │   └── studio/
│   ├── lib/
│   │   └── studio/
│   ├── types/
│   │   └── studio/
│   └── services/
├── helm/
├── k8s/
├── docs/
└── studio/
```

The coding agent must inspect existing conventions before creating exact paths.

Use the repository's existing conventions when they differ from the example.

7.2 Feature flag

Studio must be optional.

Example environment variable:

```
SYNKORA_STUDIO_ENABLED=true
```

Backend configuration:

```
studio_enabled: bool = False
```

Frontend navigation should display Studio only when enabled and authorised.

7.3 Bounded module

Studio owns:

- Product entities
- Product artefacts
- Artefact versions
- Relationships
- Canvas views
- Specifications
- Product templates
- Reviews and approvals
- Product quality rules
- Studio-specific agent orchestration
- Studio external mappings

Synkora Core continues to own:

- Users
- Authentication
- Workspaces
- Memberships
- Base roles where applicable
- Agents
- Models
- Conversations
- Knowledge bases
- Retrieval
- Tools
- Integrations and credentials
- Workflow execution
- Background workers
- Usage tracking
- Audit infrastructure

- Observability

7.4 Dependency direction

Studio may depend on Synkora Core.

Synkora Core must not depend on Studio.

Studio → Core

Core x→ Studio

Where Core needs to expose an extension point, define a generic interface rather than adding product-specific behaviour.

8. Domain Model

8.1 Workspace

Use the existing Synkora workspace or tenant model.

Every Studio resource must include `workspace_id`.

Workspace isolation must be enforced at query and authorisation levels.

8.2 Product

A top-level Studio project.

Fields:

```
id
workspace_id
name
slug
description
vision
product_type
lifecycle_stage
status
owner_user_id
owner_team_id
template_id
default_knowledge_base_ids
settings
created_by
```

```
created_at
updated_at
archived_at
```

Suggested statuses:

```
draft
discovery
definition
engineering_ready
in_delivery
released
archived
```

8.3 Artefact

A structured product-development object.

Common fields:

```
id
workspace_id
product_id
type
schema_version
title
summary
content
status
lifecycle_stage
priority
owner_user_id
owner_team_id
source_type
source_metadata
ai_metadata
current_version
approval_status
created_by
created_at
updated_at
archived_at
```

8.4 Artefact version

Immutable version of an artefact.

Fields:

```
id
artefact_id
version_number
title
summary
content
status
owner_user_id
approval_status
change_summary
change_source
created_by
created_at
```

`change_source` values:

```
human
ai_generation
ai_suggestion
external_sync
restore
migration
```

8.5 Relationship

A typed directed relationship between two artefacts.

Fields:

```
id
workspace_id
product_id
source_artefact_id
target_artefact_id
relationship_type
description
metadata
source_type
confidence
approval_status
created_by
created_at
updated_at
deleted_at
```

8.6 Specification

A named collection or projection of artefacts.

Fields:

```
id
workspace_id
product_id
name
type
description
projection_config
status
current_version
created_by
created_at
updated_at
```

Specification types:

```
product_brief
prd
technical_design
security_review
test_plan
engineering_handoff
release_readiness
custom
```

8.7 Specification version

Immutable published snapshot.

Fields:

```
id
specification_id
version_number
snapshot
release_notes
published_by
published_at
```

The snapshot must capture:

- Included artefact IDs and versions
- Included relationship IDs
- Projection configuration
- Approval state
- Quality score
- Publication metadata

8.8 Canvas view

Fields:

```
id
workspace_id
product_id
name
description
view_type
filter_config
layout_config
is_default
visibility
created_by
created_at
updated_at
```

8.9 Comment

Use the existing comment system if suitable; otherwise create a Studio module.

Fields:

```
id
workspace_id
product_id
artefact_id
parent_comment_id
body
selection_reference
status
created_by
created_at
updated_at
resolved_by
resolved_at
```

8.10 Review request

Fields:

```
id
workspace_id
product_id
resource_type
resource_id
requested_by
assigned_reviewer_id
review_type
status
due_at
message
created_at
completed_at
```

8.11 Approval

Use Synkora's existing approval infrastructure where reusable.

Studio-specific fields:

```
id
review_request_id
resource_type
resource_id
resource_version
decision
comment
decided_by
decided_at
```

Decisions:

```
approved
changes_requested
rejected
cancelled
```

8.12 Quality finding

Fields:


```
id
workspace_id
product_id
artefact_id
rule_code
category
severity
title
description
evidence
suggested_action
status
detected_by
detected_at
resolved_by
resolved_at
```

8.13 External link

Fields:

```
id
workspace_id
product_id
artefact_id
provider
external_type
external_id
external_url
metadata
sync_direction
sync_status
last_synced_at
created_at
updated_at
```

8.14 Generation run

Fields:

```
id
workspace_id
product_id
workflow_type
status
current_stage
requested_by
```

```
input
plan
result_summary
error
started_at
completed_at
```

8.15 Generation stage

Fields:

```
id
generation_run_id
stage_key
sequence
agent_id
status
input_artefact_ids
output_artefact_ids
warnings
error
started_at
completed_at
```

9. Database Design

9.1 PostgreSQL

Use PostgreSQL as the primary database.

Do not introduce a graph database in the MVP.

Suggested table names:

```
studio_products
studio_artefacts
studio_artefact_versions
studio_relationships
studio_specifications
studio_specification_versions
studio_canvas_views
studio_comments
studio_review_requests
studio_approvals
studio_quality_findings
```

```
studio_external_links
studio_generation_runs
studio_generation_stages
studio_templates
studio_template_versions
```

If the existing architecture supports PostgreSQL schemas cleanly, use:

```
studio.products
studio.artefacts
studio.artefact_versions
...
```

9.2 Content storage

Store artefact-specific fields in a JSONB `content` column.

Every artefact must also have:

- A type.
- A schema version.
- Runtime schema validation.
- Database-level indexes on commonly queried fields.

9.3 Required indexes

At minimum:

```
studio_artefacts(workspace_id, product_id)
studio_artefacts(product_id, type)
studio_artefacts(product_id, status)
studio_artefacts(product_id, owner_user_id)
studio_artefacts USING GIN(content)
studio_relationships(product_id, source_artefact_id)
studio_relationships(product_id, target_artefact_id)
studio_relationships(product_id, relationship_type)
studio_comments(artefact_id, status)
studio_quality_findings(product_id, status, severity)
studio_external_links(provider, external_id)
studio_generation_runs(product_id, status)
```

9.4 Concurrency

Add optimistic concurrency to mutable resources.

API updates must require either:

```
If-Match: <version>
```

or a request field:

```
{  
  "expected_version": 4  
}
```

Return HTTP `409 Conflict` when the current version differs.

9.5 Soft deletion

Products, artefacts, relationships and comments should use soft deletion or archival where practical.

Approved artefact versions must never be physically overwritten.

9.6 Migration requirements

Create forward and rollback migration logic following existing repository conventions.

Migrations must:

1. Create Studio tables.
2. Create constraints and indexes.
3. Seed initial artefact schemas.
4. Seed relationship types.
5. Seed the default product template.
6. Avoid modifying unrelated Synkora tables unless required for a generic extension point.

10. Artefact Type Catalogue

10.1 MVP types

Implement these artefacts first:

```
idea  
problem_statement  
persona  
business_objective  
product_metric  
requirement  
non_functional_requirement  
feature  
user_story
```

```
acceptance_criterion
architecture_component
api_resource
api_operation
data_entity
security_requirement
test_strategy
test_case
delivery_task
risk
decision
```

10.2 Later types

```
research_question
research_finding
competitor
market_trend
assumption
constraint
journey
user_flow
screen
design_reference
database
data_field
event
queue
integration
threat
security_control
milestone
epic
pull_request
build
deployment
environment
monitor
alert
slo
runbook
release
incident
product_outcome
```

11. Artefact Schemas

All schemas must use JSON Schema or the repository's equivalent typed schema system.

11.1 Common envelope

```
{
  "type": "feature",
  "schema_version": 1,
  "title": "Automated meeting preparation",
  "summary": "Generate a customer briefing before a meeting.",
  "content": {},
  "status": "draft",
  "priority": "medium",
  "owner_user_id": null,
  "owner_team_id": null
}
```

11.2 Idea schema

```
{
  "description": "Natural-language product idea",
  "target_users": [],
  "problem_hypothesis": "",
  "expected_outcomes": [],
  "known_constraints": [],
  "existing_product": false,
  "source_references": []
}
```

11.3 Problem statement schema

```
{
  "problem": "",
  "affected_users": [],
  "current_state": "",
  "desired_state": "",
  "evidence": [],
  "assumptions": [],
  "out_of_scope": []
}
```

11.4 Persona schema

```
{
  "name": "",
  "role": "",
  "description": "",
  "goals": [],
  "pain_points": [],
  "behaviours": [],
  "needs": [],
  "constraints": [],
  "evidence": []
}
```

11.5 Business objective schema

```
{
  "objective": "",
  "rationale": "",
  "desired_outcome": "",
  "measurement_window": "",
  "baseline": null,
  "target": null,
  "unit": null,
  "risks": []
}
```

11.6 Product metric schema

```
{
  "name": "",
  "description": "",
  "metric_type": "outcome",
  "calculation": "",
  "data_source": "",
  "baseline": null,
  "target": null,
  "unit": "",
  "measurement_frequency": "",
  "guardrail": false
}
```

11.7 Requirement schema

```
{
  "statement": "",
  "rationale": "",
  "requirement_type": "functional",
  "priority_method": "must_should_could_wont",
  "priority_value": "must",
  "source": "",
  "constraints": [],
  "dependencies": [],
  "validation_method": "",
  "out_of_scope": []
}
```

11.8 Feature schema

```
{
  "problem": "",
  "value_proposition": "",
  "target_personas": [],
  "scope": {
    "included": [],
    "excluded": []
  },
  "success_metrics": [],
  "dependencies": [],
  "risks": [],
  "release_strategy": ""
}
```

11.9 User story schema

```
{
  "persona": "",
  "capability": "",
  "benefit": "",
  "story": "As a ..., I want ..., so that ...",
  "notes": [],
  "dependencies": []
}
```


11.10 Acceptance criterion schema

```
{
  "criterion": "",
  "format": "given_when_then",
  "given": "",
  "when": "",
  "then": "",
  "negative_case": false,
  "automatable": null
}
```

11.11 Architecture component schema

```
{
  "component_type": "service",
  "responsibility": "",
  "technology": "",
  "interfaces": [],
  "dependencies": [],
  "scalability_requirements": [],
  "reliability_requirements": [],
  "security_considerations": [],
  "deployment_notes": []
}
```

11.12 API resource schema

```
{
  "name": "",
  "description": "",
  "base_path": "",
  "authentication": "",
  "authorisation": "",
  "versioning": "",
  "data_classification": ""
}
```

11.13 API operation schema

```
{
  "method": "POST",
  "path": "",
  "summary": "",
  "request_schema": {},
}
```

```

    "response_schema": {},
    "error_responses": [],
    "authentication": "",
    "authorisation_rules": [],
    "idempotency": "",
    "rate_limit": "",
    "audit_events": []
}

```

11.14 Data entity schema

```

{
  "name": "",
  "description": "",
  "fields": [],
  "primary_key": "",
  "relationships": [],
  "classification": "",
  "retention": "",
  "residency": "",
  "encryption": "",
  "audit_requirements": []
}

```

11.15 Security requirement schema

```

{
  "requirement": "",
  "category": "",
  "threat": "",
  "control": "",
  "verification_method": "",
  "severity": "",
  "owner": ""
}

```

11.16 Test case schema

```

{
  "title": "",
  "test_type": "functional",
  "preconditions": [],
  "steps": [],
  "expected_result": "",
  "automation_candidate": null,
  "environment": "",
}

```

```
"test_data": []  
}
```

11.17 Risk schema

```
{  
  "description": "",  
  "category": "",  
  "likelihood": "medium",  
  "impact": "medium",  
  "score": null,  
  "mitigation": "",  
  "owner": "",  
  "status": "open"  
}
```

11.18 Decision schema

```
{  
  "question": "",  
  "decision": "",  
  "context": "",  
  "options_considered": [],  
  "rationale": "",  
  "consequences": [],  
  "decision_status": "proposed",  
  "decision_date": null  
}
```

12. Relationship Types

Implement a controlled relationship registry.

12.1 MVP relationship types

```
DERIVED_FROM  
SUPPORTS  
MEASURED_BY  
TARGETS  
CONTAINS  
DEFINED_BY  
DEPENDS_ON  
CONFLICTS_WITH  
IMPLEMENTS
```

```
IMPLEMENTED_BY  
EXPOSED_BY  
STORES_DATA_IN  
VERIFIED_BY  
MITIGATED_BY  
ASSIGNED_TO  
TRACKED_BY
```

12.2 Relationship constraints

Examples:

```
business_objective → MEASURED_BY → product_metric  
feature → SUPPORTS → business_objective  
feature → TARGETS → persona  
feature → DEFINED_BY → requirement  
feature → CONTAINS → user_story  
user_story → VERIFIED_BY → acceptance_criterion  
architecture_component → IMPLEMENTS → feature  
api_operation → EXPOSED_BY → architecture_component  
api_operation → STORES_DATA_IN → data_entity  
test_case → VERIFIED_BY → acceptance_criterion  
security_requirement → MITIGATED_BY → risk
```

The relationship registry should declare:

- Allowed source types.
- Allowed target types.
- Whether multiple edges are allowed.
- Whether cycles are allowed.
- Whether the relationship is directional.
- Its inverse label.
- Its display style.

12.3 Graph validation

Reject invalid relationships with HTTP `422`.

Prevent accidental cross-workspace and cross-product relationships unless an explicit shared-resource feature is later introduced.

13. API Specification

All endpoints must follow existing Synkora API conventions.

Use `/api/studio` or the repository's established versioned prefix.

13.1 Products

```
POST    /api/studio/products
GET      /api/studio/products
GET      /api/studio/products/{product_id}
PATCH   /api/studio/products/{product_id}
DELETE   /api/studio/products/{product_id}
POST     /api/studio/products/{product_id}/archive
POST     /api/studio/products/{product_id}/restore
```

Create product request:

```
{
  "name": "AI CRM",
  "description": "AI-powered CRM for financial institutions",
  "template_id": "default-product",
  "idea": {
    "description": "Build an AI CRM...",
    "target_users": ["Relationship managers"],
    "known_constraints": []
  },
  "knowledge_base_ids": []
}
```

Create product response:

```
{
  "product": {},
  "idea_artefact": {},
  "recommended_generation_plan": {}
}
```

13.2 Artefacts

```
POST    /api/studio/products/{product_id}/artefacts
GET      /api/studio/products/{product_id}/artefacts
GET      /api/studio/artefacts/{artefact_id}
PATCH   /api/studio/artefacts/{artefact_id}
DELETE   /api/studio/artefacts/{artefact_id}
GET      /api/studio/artefacts/{artefact_id}/versions
GET      /api/studio/artefacts/{artefact_id}/versions/{version}
POST     /api/studio/artefacts/{artefact_id}/restore/{version}
POST     /api/studio/artefacts/{artefact_id}/duplicate
```

List filters:

```
type
status
owner_user_id
owner_team_id
approval_status
lifecycle_stage
search
limit
cursor
```

13.3 Relationships

```
POST   /api/studio/products/{product_id}/relationships
GET    /api/studio/products/{product_id}/relationships
DELETE /api/studio/relationships/{relationship_id}
```

Create request:

```
{
  "source_artefact_id": "art_feature_1",
  "target_artefact_id": "art_objective_1",
  "relationship_type": "SUPPORTS",
  "description": ""
}
```

13.4 Graph

```
GET /api/studio/products/{product_id}/graph
GET /api/studio/products/{product_id}/graph/subgraph
```

Graph query parameters:

```
artefact_types
relationship_types
root_artefact_id
depth
statuses
owners
include_archived
```

Response:

```
{
  "nodes": [],
```

```

    "edges": [],
    "metadata": {
      "total_nodes": 0,
      "total_edges": 0,
      "truncated": false
    }
  }
}

```

13.5 Canvas views

```

POST   /api/studio/products/{product_id}/canvas-views
GET    /api/studio/products/{product_id}/canvas-views
GET    /api/studio/canvas-views/{view_id}
PATCH /api/studio/canvas-views/{view_id}
DELETE /api/studio/canvas-views/{view_id}
POST   /api/studio/canvas-views/{view_id}/set-default

```

13.6 Generation

```

POST /api/studio/products/{product_id}/generation-runs
GET  /api/studio/generation-runs/{run_id}
POST /api/studio/generation-runs/{run_id}/approve-stage
POST /api/studio/generation-runs/{run_id}/reject-stage
POST /api/studio/generation-runs/{run_id}/retry
POST /api/studio/generation-runs/{run_id}/cancel

```

Create request:

```

{
  "workflow_type": "idea_to_engineering_handoff",
  "mode": "review_each_stage",
  "selected_stages": [
    "problem_definition",
    "personas",
    "objectives",
    "requirements",
    "features",
    "stories",
    "architecture",
    "api",
    "data",
    "testing"
  ],
  "knowledge_base_ids": [],
  "generation_depth": "standard"
}

```

Modes:

```
review_each_stage
review_at_checkpoints
automatic_draft
```

No mode may automatically publish or approve content.

13.7 AI actions

```
POST /api/studio/artefacts/{artefact_id}/ai/improve
POST /api/studio/artefacts/{artefact_id}/ai/expand
POST /api/studio/artefacts/{artefact_id}/ai/suggest-links
POST /api/studio/artefacts/{artefact_id}/ai/check
POST /api/studio/products/{product_id}/ai/query
POST /api/studio/products/{product_id}/ai/impact-analysis
```

All AI mutation endpoints return suggestions unless the caller explicitly accepts generated drafts.

13.8 Suggestions

```
GET /api/studio/products/{product_id}/suggestions
POST /api/studio/suggestions/{suggestion_id}/accept
POST /api/studio/suggestions/{suggestion_id}/reject
```

A suggestion must contain:

```
suggestion_type
target_resource
proposed_patch
reason
source_agent
confidence
evidence
created_at
```

13.9 Specifications

```
POST /api/studio/products/{product_id}/specifications
GET /api/studio/products/{product_id}/specifications
GET /api/studio/specifications/{specification_id}
GET /api/studio/specifications/{specification_id}/render
POST /api/studio/specifications/{specification_id}/publish
GET /api/studio/specifications/{specification_id}/versions
```


Render formats:

```
json
html
markdown
```

PDF export may be added after MVP.

13.10 Reviews and approvals

```
POST /api/studio/resources/{resource_type}/{resource_id}/review-requests
GET /api/studio/review-requests
GET /api/studio/review-requests/{review_id}
POST /api/studio/review-requests/{review_id}/approve
POST /api/studio/review-requests/{review_id}/request-changes
POST /api/studio/review-requests/{review_id}/reject
```

13.11 Comments

```
POST /api/studio/artefacts/{artefact_id}/comments
GET /api/studio/artefacts/{artefact_id}/comments
PATCH /api/studio/comments/{comment_id}
DELETE /api/studio/comments/{comment_id}
POST /api/studio/comments/{comment_id}/resolve
POST /api/studio/comments/{comment_id}/reopen
```

13.12 Quality

```
POST /api/studio/products/{product_id}/quality-runs
GET /api/studio/products/{product_id}/quality
GET /api/studio/products/{product_id}/quality-findings
POST /api/studio/quality-findings/{finding_id}/resolve
POST /api/studio/quality-findings/{finding_id}/dismiss
```

13.13 External context export

```
POST /api/studio/products/{product_id}/exports/coding-agent
POST /api/studio/features/{feature_id}/exports/coding-agent
```

Request:

```
{
  "format": "markdown",
```

```
"include": [  
  "requirements",  
  "acceptance_criteria",  
  "architecture",  
  "api",  
  "data",  
  "security",  
  "tests"  
],  
"target_agent": "claude-code"  
}
```

Output should be deterministic and structured.

14. Events

Publish domain events through the existing event or background-work mechanism.

Required events:

```
studio.product.created  
studio.product.updated  
studio.product.archived  
studio.artefact.created  
studio.artefact.updated  
studio.artefact.archived  
studio.artefact.version.created  
studio.artefact.approved  
studio.relationship.created  
studio.relationship.deleted  
studio.generation.started  
studio.generation.stage.completed  
studio.generation.stage.failed  
studio.generation.completed  
studio.review.requested  
studio.review.completed  
studio.specification.published  
studio.quality.completed  
studio.quality.finding.created  
studio.external_link.created
```

Event payloads must include:

```
event_id  
event_type  
workspace_id
```

```
product_id
actor_type
actor_id
resource_type
resource_id
timestamp
correlation_id
```

15. AI Agent Design

Studio presents one primary interface:

Studio Copilot

Behind it, Synkora orchestrates specialist agents.

15.1 Specialist agents

Product Strategist Agent

Inputs:

- Idea
- Existing product context
- Selected knowledge bases

Outputs:

- Problem statement
- Assumptions
- Constraints
- Clarifying questions
- Initial product scope

Research Agent

Inputs:

- Problem statement
- Research questions
- Internal knowledge
- Approved external tools

Outputs:

- Research findings
- Evidence

- Competitor summaries
- Uncertainties

External research must be optional and policy-controlled.

Product Manager Agent

Outputs:

- Personas
- Objectives
- Metrics
- Requirements
- Features
- User stories
- Acceptance criteria
- Risks

UX Agent

Outputs:

- Journeys
- User flows
- Screen definitions
- Required states
- Accessibility requirements

UX generation may be deferred from the initial MVP UI, but the agent contract should be extensible.

Architect Agent

Outputs:

- Architecture components
- Responsibilities
- Dependencies
- Interfaces
- Technical decisions
- Non-functional requirements

API Designer Agent

Outputs:

- API resources
- API operations
- Request and response schemas
- Error cases
- Authentication and authorisation rules

Data Architect Agent

Outputs:

- Entities
- Fields
- Relationships
- Classifications
- Retention requirements
- Migration considerations

Security Agent

Outputs:

- Threats
- Security requirements
- Controls
- Security test recommendations
- Missing authorisation or data-protection requirements

QA Agent

Outputs:

- Test strategy
- Test cases
- Acceptance-criteria coverage
- Negative and edge cases
- Non-functional tests

Delivery Planner Agent

Outputs:

- Epics
- Tasks
- Dependencies
- Milestones
- Suggested implementation sequence

Governance Agent

Outputs:

- Quality findings
- Contradictions
- Missing links
- Readiness assessment
- Required approvals

Change Impact Agent

Inputs:

- Proposed artefact change
- Product subgraph

Outputs:

- Affected artefacts
- Affected external objects
- Risk summary
- Proposed follow-up edits
- Required reviewers

15.2 Agent output contract

Every specialist agent must return:

```
{
  "artefacts": [],
  "relationships": [],
  "questions": [],
  "warnings": [],
  "sources": [],
  "summary": ""
}
```

Each artefact output:

```
{
  "temporary_id": "tmp_1",
  "type": "feature",
  "schema_version": 1,
  "title": "",
  "summary": "",
  "content": {},
  "confidence": 0.82,
  "source_references": [],
  "warnings": []
}
```

Each relationship output:

```
{
  "source_ref": "tmp_1",
  "target_ref": "existing_or_temporary_id",
  "relationship_type": "SUPPORTS",
}
```

```
"confidence": 0.86,  
"reason": ""  
}
```

15.3 Validation pipeline

For every agent execution:

```
Agent produces structured output  
↓  
Parse output  
↓  
Validate common envelope  
↓  
Validate artefact type schema  
↓  
Validate relationship rules  
↓  
Validate referenced artefacts  
↓  
Attempt one structured repair when invalid  
↓  
Store draft output or mark stage failed
```

Do not store invalid artefacts.

15.4 AI metadata

Every AI-generated artefact version must store:

```
agent_id  
agent_version  
model_provider  
model_name  
prompt_template_id  
prompt_template_version  
input_artefact_ids  
knowledge_source_ids  
tool_execution_ids  
confidence  
generation_run_id  
generated_at
```

15.5 Source discipline

Agents must distinguish:

```
confirmed_fact
source_supported_inference
user_assumption
agent_suggestion
unknown
```

Market or competitor claims must include references when external research is enabled.

16. AI Workflow Definitions

16.1 Idea-to-product-definition

1. Read idea
2. Generate clarifying questions
3. User answers or accepts assumptions
4. Generate problem statement
5. User reviews
6. Generate personas
7. Generate business objectives
8. Generate metrics
9. Generate requirements
10. Generate features
11. Run product-definition quality checks
12. Request product-owner review

16.2 Feature-to-engineering-handoff

1. Select approved feature
2. Load linked requirements and stories
3. Generate missing acceptance criteria
4. Generate non-functional requirements
5. Generate architecture components
6. Generate API outline
7. Generate data entities
8. Generate security requirements
9. Generate test strategy and test cases
10. Generate implementation tasks
11. Run readiness checks
12. Request engineering review

16.3 Change-impact workflow

1. Receive proposed artefact modification
2. Load incoming and outgoing relationships

3. Traverse graph to configured depth
4. Load linked external objects
5. Compare old and proposed versions
6. Identify impacted artefacts
7. Classify impact severity
8. Generate recommended changes
9. Create suggestions
10. Request relevant reviews

16.4 Quality workflow

1. Load product graph
2. Run deterministic rules
3. Run semantic consistency checks
4. Deduplicate findings
5. Calculate category scores
6. Store findings
7. Present suggested resolutions

17. Prompt Management

Do not hard-code large prompts inside route handlers or service methods.

Use Synkora's existing prompt or agent configuration where possible.

Each prompt template must have:

```
id
name
agent_type
version
system_prompt
input_template
output_schema
status
created_at
updated_at
```

Required initial prompts:

```
studio_product_strategist_v1
studio_persona_generator_v1
studio_objective_generator_v1
studio_requirement_generator_v1
```

```
studio_feature_generator_v1
studio_story_generator_v1
studio_architect_v1
studio_api_designer_v1
studio_data_architect_v1
studio_security_reviewer_v1
studio_qa_v1
studio_governance_v1
studio_change_impact_v1
```

Prompt behaviour requirements:

- Use supplied context only.
- Explicitly list assumptions.
- Ask questions when critical information is missing.
- Never claim unsupported facts.
- Return only schema-compatible structured output.
- Avoid duplicate artefacts.
- Reference source artefacts by ID.
- Preserve approved terminology.
- Include warnings for low-confidence content.

18. Product Templates

A product template defines:

```
Lifecycle stages
Required artefact types
Optional artefact types
Generation workflow
Approval policies
Quality rules
Default agents
Default document projections
Default canvas views
```

18.1 Initial default template

```
General Software Product
```

Required artefacts:

```
idea
problem_statement
persona
```

```
business_objective
product_metric
requirement
feature
user_story
acceptance_criterion
architecture_component
api_resource or explicit no-API decision
data_entity or explicit no-data decision
test_strategy
risk
```

18.2 Suggested Deriv templates

Later additions:

```
Existing Product Enhancement
Payment Integration
AI Feature
Internal Platform
Security Remediation
Regulatory Change
Operational Improvement
```

19. Quality Engine

19.1 Deterministic rules first

Implement deterministic quality rules before LLM-based quality analysis.

Initial rules:

```
STUDIO-Q001 Objective has no linked metric
STUDIO-Q002 Feature has no linked objective
STUDIO-Q003 Feature has no requirement
STUDIO-Q004 Feature has no user story
STUDIO-Q005 User story has no acceptance criterion
STUDIO-Q006 Requirement has no validation method
STUDIO-Q007 Architecture component has no linked feature
STUDIO-Q008 API operation has no authentication definition
STUDIO-Q009 API operation has no authorisation rules
STUDIO-Q010 Data entity has no classification
STUDIO-Q011 Feature has no test coverage
STUDIO-Q012 High-priority risk has no mitigation
STUDIO-Q013 Approved artefact has unresolved critical comments
```

```
STUDIO-Q014 Specification contains unapproved required artefacts
STUDIO-Q015 Artefact references archived dependency
STUDIO-Q016 Required artefact type is missing
STUDIO-Q017 Duplicate title within artefact type
STUDIO-Q018 Product metric has no linked objective
STUDIO-Q019 Security requirement has no verification method
STUDIO-Q020 Engineering handoff is incomplete
```

19.2 Semantic checks

Add after deterministic rules:

- Contradictory requirements.
- Duplicate features with different wording.
- Acceptance criteria that are not testable.
- Architecture that does not satisfy non-functional requirements.
- Security controls that do not address the named threat.
- Metrics that do not measure the objective.
- Unsupported assumptions presented as facts.

19.3 Readiness scores

Calculate:

```
Product definition
Traceability
Testability
Technical readiness
Security readiness
Delivery readiness
Overall readiness
```

Scores must be explainable.

Example:

```
{
  "overall": 74,
  "categories": {
    "product_definition": 92,
    "traceability": 81,
    "testability": 61,
    "technical_readiness": 72,
    "security_readiness": 54
  },
  "blocking_findings": 3
}
```

Do not allow a score alone to approve a specification.

20. Document Projection Engine

20.1 Principle

Documents are projections of graph data.

Do not create an independent, editable document body that can diverge from the graph.

20.2 PRD projection

Sections:

1. Executive Summary
2. Problem Statement
3. Product Vision
4. Target Personas
5. Business Objectives
6. Success Metrics
7. Scope
8. Requirements
9. Features
10. User Stories
11. Acceptance Criteria
12. Risks
13. Dependencies
14. Open Decisions
15. Approval Status

20.3 Engineering handoff projection

Sections:

1. Feature Summary
2. Business Context
3. Requirements
4. User Stories
5. Acceptance Criteria
6. Non-Functional Requirements
7. Architecture
8. APIs
9. Data Model
10. Security Requirements
11. Testing
12. Delivery Tasks

- 13. Risks
- 14. Open Decisions
- 15. Definition of Ready

20.4 Rendering

Support:

- HTML for the application.
- Markdown for export.
- JSON for automation.

Every rendered section should include source artefact identifiers internally.

Editing a section in the UI must update the underlying artefact, not a generated document string.

21. Frontend Information Architecture

21.1 Global navigation

When Studio is enabled:

Dashboard
Agents
Knowledge
Integrations
Studio
Settings

21.2 Studio navigation

Studio Home
Products
Reviews
Agent Activity
Templates
Studio Settings

21.3 Product navigation

Overview
Canvas
PRD
Features

Architecture
APIs
Data
Tests
Risks
Decisions
Reviews
Activity
Settings

Hide empty advanced sections when they are not relevant, but keep them discoverable.

22. Frontend Screens

22.1 Studio Home

Display:

- Recent products.
- Products requiring attention.
- Assigned reviews.
- Active generation runs.
- Quality alerts.
- Recent Studio activity.

Actions:

New Product
Import Existing Spec
Open Product
Review Assigned Item

22.2 New Product Wizard

Steps:

Step 1: Product idea

Fields:

Product name
What are you building?
Who is it for?
What problem does it solve?

Expected outcome
Known constraints

Only product name and idea description are required.

Step 2: Product context

Options:

New product
Existing product
Product enhancement
Internal platform
AI feature

Step 3: Knowledge

Allow selection of existing Synkora knowledge bases.

Step 4: Template

Select product template.

Step 5: Generation mode

Generate product definition
Generate MVP specification
Start with blank canvas
Import existing specification

22.3 Generation Plan

Display stages as cards or vertical steps.

Each stage shows:

Name
Description
Agent
Dependencies
Expected outputs
Status
Warnings
Review requirement

Actions:

Start
Pause
Cancel
Retry
Approve Stage
Request Changes
Skip Optional Stage

Progress should stream using existing realtime infrastructure.

22.4 Product Overview

Display:

Product summary
Lifecycle stage
Overall readiness
Objectives
Key metrics
Top risks
Pending decisions
Pending reviews
Recent activity
Connected external systems

Primary actions:

Continue Generation
Open Canvas
Open PRD
Run Quality Check
Generate Engineering Handoff

22.5 Canvas

Use React Flow or a compatible graph library unless the repository already has a preferred equivalent.

Required capabilities:

- Pan.
- Zoom.
- Fit view.
- Drag nodes.
- Multi-select.
- Connect nodes.
- Delete permitted edges.

- Auto-layout.
- Minimap.
- Search.
- Type filters.
- Status filters.
- Owner filters.
- Relationship filters.
- Depth control.
- Saved views.
- Node detail drawer.
- Context menus.
- Keyboard shortcuts.
- Loading and empty states.
- Read-only mode.

MVP performance target:

500 visible nodes
1,500 visible relationships

Use viewport-based rendering and avoid unnecessary full-graph rerenders.

22.6 Node card

Display:

Type
Title
Status
Approval status
Owner
Risk or warning indicator
Comment count
Relationship count
AI-generated indicator

22.7 Node detail drawer

Tabs:

Details
Relationships
Comments
Versions
AI

Activity
External Links

AI actions:

Improve
Expand
Find Gaps
Suggest Relationships
Generate Children
Analyse Impact

Every AI action must preview changes before applying them.

22.8 PRD View

Display structured sections in a document-style layout.

Features:

- Inline artefact editing.
- Collapsible sections.
- Comments.
- Status badges.
- Approval state.
- Change history.
- Export.
- Publish version.

22.9 Feature Detail

This is a key product screen.

Sections:

Summary
Business Value
Target Personas
Requirements
User Stories
Acceptance Criteria
Architecture
APIs
Data
Security
Tests
Delivery
Metrics

Risks
Decisions
External Links

Display a traceability path:

Objective → Requirement → Feature → Story → API → Service → Test → Metric

22.10 Review Centre

Views:

Assigned to Me
Requested by Me
Completed
All Product Reviews

Review screen displays:

- Resource.
- Version.
- Field-level change.
- Relationship changes.
- Comments.
- Quality findings.
- Approve, request changes or reject.

22.11 Agent Activity

Display:

- Active runs.
- Run history.
- Stage status.
- Model used.
- Tool calls.
- Duration.
- Token or cost information where available.
- Warnings.
- Errors.
- Retry controls.

Reuse Synkora observability components where possible.

22.12 Product Settings

Configure:

```
Product members
Product roles
Default agents
Knowledge bases
AI models
Generation behaviour
Required approvals
Quality thresholds
External integrations
Archival
```

23. State Management and Frontend Data

Use the existing Synkora frontend data-fetching convention.

Requirements:

- Server-side fetch where useful.
- Client cache for interactive graph state.
- Optimistic updates with rollback.
- Explicit loading, empty and error states.
- Cursor pagination for lists.
- WebSocket event handling for generation progress and collaboration events.
- URL-addressable filters and selected artefacts where practical.

Suggested Studio frontend types:

```
StudioProduct
StudioArtefact
StudioRelationship
StudioGraph
StudioCanvasView
StudioGenerationRun
StudioGenerationStage
StudioReviewRequest
StudioQualityFinding
StudioSpecification
```

24. Permissions

24.1 Roles

Use existing workspace roles where possible and add product-level assignments.

Suggested Studio permissions:

```
studio.product.create
studio.product.read
studio.product.update
studio.product.archive

studio.artefact.create
studio.artefact.read
studio.artefact.update
studio.artefact.archive
studio.artefact.approve

studio.relationship.create
studio.relationship.delete

studio.generation.run
studio.generation.approve_stage

studio.comment.create
studio.comment.resolve

studio.review.request
studio.review.complete

studio.specification.publish

studio.template.manage
studio.settings.manage
studio.audit.read
```

24.2 Suggested role mapping

Workspace administrator

All Studio permissions.

Product administrator

All permissions within assigned products except workspace-level configuration.

Editor

Create and update artefacts, relationships and comments; run generation.

Reviewer

Read, comment and complete assigned reviews.

Viewer

Read approved and shared content.

24.3 Product access

Add product membership or product access records if workspace-wide access is too broad.

Every API request must verify:

```
Workspace membership
Studio feature access
Product access
Action permission
Data-classification restrictions where configured
```

25. Review and Approval Policies

25.1 Approval applies to versions

Approval must reference a specific artefact or specification version.

Editing approved content creates a new draft version.

The previous approved version remains approved until the new version is reviewed.

25.2 Default policies

```
Problem statement:
Product owner approval

PRD publication:
Product owner approval

Technical design:
Engineering lead approval

High-severity security requirement:
Security reviewer approval

Engineering handoff:
Product owner and engineering lead approval
```

Make policies template-configurable.

25.3 Approval states

```
not_required  
draft  
in_review  
changes_requested  
approved  
rejected  
superseded
```

26. Versioning

26.1 Artefact version creation

Create a new version when:

- A user updates meaningful content.
- An accepted AI suggestion changes content.
- An external synchronisation changes mapped content.
- A previous version is restored.

Do not create versions for ephemeral UI changes such as canvas position.

26.2 Diff

Provide field-level JSON diffs rendered in a human-readable format.

Show:

```
Added fields  
Removed fields  
Modified fields  
Relationship additions  
Relationship removals
```

26.3 Specification publication

Publishing creates an immutable snapshot.

Requirements:

- No unresolved blocking quality findings.
- Required artefacts exist.
- Required approvals are complete.
- User confirms release notes.

Administrators may configure whether warnings block publication.

27. Collaboration

27.1 MVP

Implement:

- Comments.
- Replies.
- Mentions.
- Comment resolution.
- Review requests.
- Assignment.
- Activity feed.
- Realtime generation updates.

27.2 Post-MVP

Add:

- Realtime simultaneous field editing.
- Presence indicators.
- Collaborative cursors.
- Specification branches.
- Merge proposals.

Do not block the MVP on multiplayer document editing.

28. Coding-Agent Export

A major MVP capability is generating implementation context for Codex or Claude Code.

28.1 Feature export structure

```
# Implementation Context

## Product
## Business Objective
## Feature
## Requirements
## User Stories
## Acceptance Criteria
## Non-Functional Requirements
## Architecture
```

```
## API Contract
## Data Model
## Security Requirements
## Test Requirements
## Dependencies
## Risks
## Open Decisions
## Existing Repository Context
## Implementation Constraints
## Definition of Done
```

28.2 Export rules

- Include approved artefacts by default.
- Clearly label drafts.
- Include IDs for traceability.
- Include exact API and schema definitions.
- Include unresolved questions.
- Include explicit non-goals.
- Avoid including unrelated product context.
- Support Markdown and JSON.
- Never include credentials or secrets.

28.3 Agent implementation task format

Studio should optionally generate:

```
Goal
Repository scope
Files likely affected
Required behaviour
Acceptance criteria
Required tests
Security constraints
Observability requirements
Migration requirements
Commands to validate
Out-of-scope changes
```

This is the bridge between Studio and coding agents.

29. Integrations

29.1 MVP: GitHub

Reuse Synkora's current GitHub integration where available.

Capabilities:

- Connect repository.
- Associate product with repositories.
- Associate feature with repository.
- Link issue.
- Link pull request.
- Link commit.
- Display basic status.
- Export implementation context.
- Optionally create an issue after human approval.

Do not build automatic code modification into Studio MVP.

29.2 MVP: Slack

Capabilities:

- Share product, artefact or review link.
- Notify assigned reviewer.
- Notify generation completion.
- Create an idea from an approved message workflow later.

29.3 Optional MVP: Jira or ClickUp

Choose one based on Deriv's preferred delivery system.

Initial scope:

- Link an existing task.
- Create a task after approval.
- Display status.
- Store external URL and identifier.

29.4 Post-MVP

Figma
Datadog
GitLab
Linear
Confluence
Google Drive
CI/CD
Argo CD
Kubernetes
Sentry
BrowserStack
Feature flags

30. Security Requirements

30.1 Access security

- Reuse Synkora authentication and SSO.
- Enforce workspace isolation.
- Enforce product permissions.
- Protect all Studio routes.
- Validate ownership on every resource.
- Prevent identifier-based access across workspaces.
- Avoid trusting workspace IDs supplied only by clients.

30.2 AI security

- Apply workspace model allowlists.
- Prevent restricted content from reaching unauthorised providers.
- Record every model and tool used.
- Treat retrieved external content as untrusted.
- Protect prompts from indirect prompt injection.
- Restrict tool access by agent and workspace.
- Require human approval for external writes.
- Never expose secrets in prompts, logs or generated exports.
- Apply output validation before persistence.
- Mark AI-generated artefacts.

30.3 Data security

- Encrypt traffic.
- Use existing encryption-at-rest controls.
- Encrypt integration credentials using existing Synkora facilities.
- Support data-classification metadata.
- Log sensitive-resource access where required.
- Apply configured retention rules.
- Do not use real customer data in generated examples by default.

30.4 Audit

Audit:

- Product creation.
- Artefact creation and modification.
- AI generation.
- Suggestion acceptance or rejection.
- Relationship changes.
- Review requests.
- Approval decisions.
- Specification publication.
- External actions.
- Permission changes.

- Template changes.

30.5 Input validation

- Validate all JSON payloads.
- Sanitize rendered Markdown and HTML.
- Protect against stored XSS in comments and artefacts.
- Restrict attachment types if upload support is added.
- Validate external URLs.
- Verify webhook signatures.

31. Non-Functional Requirements

31.1 Performance

Targets under normal load:

```
Product list: p95 under 1.5 seconds
Product overview: p95 under 2 seconds
Artefact update: p95 under 750 milliseconds
Search: p95 under 2 seconds
Graph initial display, 250 nodes: under 2.5 seconds
Realtime progress update latency: under 2 seconds
```

AI generation is asynchronous and not subject to normal request latency targets.

31.2 Graph performance

MVP:

```
500 displayed nodes
1,500 displayed edges
```

The graph endpoint must support filters and depth-limited subgraphs.

Do not send an unlimited graph to the browser.

31.3 Reliability

- Background generation stages must be idempotent.
- Failed stages must be retryable.
- Completed output must survive worker restarts.
- Cancellation must stop pending stages safely.
- Partial generation results must remain reviewable.
- External integration failures must not corrupt Studio data.

- Database transactions must protect version and relationship consistency.

31.4 Availability

Studio should follow the same deployment and availability model as Synkora Core.

Failure of an AI provider must not prevent manual viewing and editing.

31.5 Accessibility

Target WCAG 2.1 AA.

Provide:

- Keyboard-accessible navigation.
- Keyboard alternative for graph relationships.
- Text labels for status indicators.
- Sufficient contrast.
- Accessible forms.
- Screen-reader descriptions.
- Reduced-motion support.

31.6 Browser support

Support the browsers already supported by Synkora.

At minimum, current stable versions of Chromium-based browsers should work.

32. Observability

Reuse existing Synkora observability.

32.1 Metrics

Add:

```
studio_products_total
studio_artefacts_total
studio_relationships_total
studio_generation_runs_total
studio_generation_stage_duration_seconds
studio_generation_failures_total
studio_ai_suggestions_total
studio_ai_suggestions_accepted_total
studio_quality_findings_total
studio_reviews_pending_total
studio_specifications_published_total
```

```
studio_graph_query_duration_seconds
studio_external_sync_failures_total
```

Labels must avoid unbounded high-cardinality identifiers.

32.2 Logs

Structured logs should include:

```
workspace_id
product_id
resource_type
resource_id
generation_run_id
stage_key
agent_id
correlation_id
operation
status
duration
```

Do not log artefact content by default.

32.3 Traces

Trace:

- HTTP request.
- Generation orchestration.
- Agent call.
- Retrieval.
- Tool execution.
- Validation.
- Database write.
- Event publication.

32.4 Dashboards

Create:

```
Studio API Health
Studio Generation Health
Studio AI Usage
Studio Quality and Reviews
Studio Integration Health
```

33. Background Jobs

Use existing Celery infrastructure.

Suggested tasks:

```
studio.run_generation_stage
studio.validate_generation_output
studio.run_quality_checks
studio.calculate_readiness
studio.generate_document_projection
studio.sync_external_link
studio.send_review_notifications
studio.run_change_impact_analysis
studio.cleanup_expired_drafts
```

Task requirements:

- Idempotency key.
- Retry policy.
- Exponential backoff.
- Maximum retries.
- Correlation ID.
- Workspace context.
- Permission context where required.
- Clear terminal failure state.

34. Realtime Events

Use existing WebSocket infrastructure.

Client events:

```
studio.generation.updated
studio.generation.stage.updated
studio.artefact.created
studio.artefact.updated
studio.relationship.created
studio.relationship.deleted
studio.comment.created
studio.review.updated
studio.quality.updated
```

Authorise every subscription against workspace and product access.

35. Search

35.1 MVP search

Use PostgreSQL full-text search plus existing semantic retrieval where appropriate.

Search:

- Product name.
- Artefact title.
- Artefact summary.
- Selected text fields in content.
- Comments.
- Decisions.

35.2 Natural-language product query

Studio Copilot must support:

- What is this product trying to achieve?
- Which features support objective X?
- Which requirements have no tests?
- What is blocking engineering readiness?
- Which API operations use customer data?
- What changes if this requirement is modified?
- Summarise unresolved decisions.

Answers must reference underlying artefacts.

36. Error Handling

Use consistent API error format.

```
{
  "error": {
    "code": "STUDIO_VERSION_CONFLICT",
    "message": "The artefact was updated by another user.",
    "details": {
      "current_version": 5
    },
    "correlation_id": "..."
  }
}
```

Initial error codes:

```
STUDIO_DISABLED
STUDIO_ACCESS_DENIED
STUDIO_PRODUCT_NOT_FOUND
STUDIO_ARTEFACT_NOT_FOUND
STUDIO_SCHEMA_INVALID
STUDIO_RELATIONSHIP_INVALID
STUDIO_VERSION_CONFLICT
STUDIO_REVIEW_REQUIRED
STUDIO_APPROVAL_REQUIRED
STUDIO_PUBLICATION_BLOCKED
STUDIO_GENERATION_FAILED
STUDIO_GENERATION_CANCELLED
STUDIO_AI_OUTPUT_INVALID
STUDIO_EXTERNAL_SYNC_FAILED
```

Never return raw provider or database errors to the user.

37. Testing Strategy

37.1 Backend unit tests

Cover:

- Schema validation.
- Relationship constraints.
- Version creation.
- Optimistic concurrency.
- Readiness scoring.
- Deterministic quality rules.
- Permission checks.
- Projection rendering.
- Suggestion acceptance.
- Generation output validation.

37.2 Backend integration tests

Cover:

- Product creation.
- Artefact CRUD.
- Relationship creation.
- Graph retrieval.
- Generation workflow.
- Review and approval.
- Publication.
- Workspace isolation.
- External-link creation.

- Background-task retries.

37.3 Frontend unit tests

Cover:

- Forms.
- Artefact cards.
- Node details.
- Filters.
- Review controls.
- Quality findings.
- Generation progress.

37.4 Frontend integration tests

Cover:

- New-product wizard.
- Product generation.
- Artefact editing.
- Relationship creation.
- PRD rendering.
- Review completion.
- Version comparison.
- Coding-agent export.

37.5 End-to-end tests

Required paths:

E2E-001 Create a product

User creates product
Idea artefact is created
Generation plan appears

E2E-002 Generate product definition

User starts generation
Stages update in realtime
Draft artefacts appear
Relationships are created
User approves a stage

E2E-003 Edit and version artefact

User edits feature
New version is created
Diff is available

E2E-004 Canvas traceability

User opens canvas
Filters to feature
Expands linked artefacts
Opens node details

E2E-005 Review

Editor requests review
Reviewer sees assignment
Reviewer requests changes
Editor updates artefact
Reviewer approves new version

E2E-006 Publish PRD

Quality rules run
Blocking issues are resolved
Required approvals exist
PRD version is published

E2E-007 Workspace isolation

User from workspace A cannot read any Studio resource from workspace B

E2E-008 Export to coding agent

User selects feature
System generates structured Markdown
Only relevant approved artefacts are included

37.6 AI evaluation tests

Create fixed evaluation fixtures.

Evaluate:

- JSON validity.
- Schema compliance.
- Duplicate rate.
- Required field coverage.
- Relationship validity.
- Unsupported factual claims.
- Testability of acceptance criteria.
- Consistency with source artefacts.

Do not rely only on snapshot tests for AI output.

38. Seed Data

Provide a development seed product:

AI CRM for Financial Institutions

Seed:

- One idea.
- One problem statement.
- Two personas.
- Two objectives.
- Three metrics.
- Four requirements.
- Three features.
- Six user stories.
- Twelve acceptance criteria.
- Three architecture components.
- Two API resources.
- Four API operations.
- Four data entities.
- Five test cases.
- Three risks.
- Two decisions.
- Relationships connecting the complete graph.

This seed should exercise canvas, PRD and feature-detail screens.

39. Deployment

39.1 Build model

Studio should compile into the existing web and API images unless there is a proven reason to separate it.

Do not create a new microservice for every Studio domain.

39.2 Configuration

Add:

```
SYNKORA_STUDIO_ENABLED  
SYNKORA_STUDIO_DEFAULT_TEMPLATE  
SYNKORA_STUDIO_MAX_GRAPH_NODES  
SYNKORA_STUDIO_EXTERNAL_RESEARCH_ENABLED  
SYNKORA_STUDIO_DEFAULT_GENERATION_MODE
```

Secrets must use existing secret-management patterns.

39.3 Helm

Add Studio values:

```
studio:  
  enabled: true  
  defaultTemplate: general-software-product  
  maxGraphNodes: 500  
  externalResearchEnabled: false
```

Do not duplicate web, API or worker deployments solely for Studio in the MVP.

Add worker resource recommendations if generation materially increases queue load.

39.4 Health

Existing health checks must continue to work.

Add optional internal readiness checks for:

- Studio migrations.
- Required prompt templates.
- Default template.
- Required agents.

39.5 Rollout

Use:

```
Feature flag  
Internal development environment  
Small Deriv pilot  
Controlled wider rollout
```

Do not enable Studio for all workspaces by default.

40. Backward Compatibility

Requirements:

- Existing Synkora routes and behaviours must remain unchanged.
 - Existing deployments with Studio disabled must not require Studio configuration.
 - Studio migrations must not alter existing data semantics.
 - Core agent usage must remain independent.
 - Shared components must remain generic.
 - Any Core change must include regression tests.
-

41. Implementation Phases

Phase 0: Repository assessment

The coding agent must first:

1. Read repository architecture documentation.
2. Inspect backend application structure.
3. Inspect database and migration conventions.
4. Inspect authentication and workspace scoping.
5. Inspect agent and workflow services.
6. Inspect WebSocket and Celery patterns.
7. Inspect frontend route and component conventions.
8. Inspect GitHub integration.
9. Inspect testing conventions.
10. Produce a short implementation map before modifying code.

Deliverable:

```
docs/studio/repository-assessment.md
```

This document should map specification concepts to actual repository files.

Phase 1: Foundation

Build:

- Feature flag.
- Product model.
- Artefact model.
- Artefact versions.
- Relationships.
- Schemas.
- CRUD APIs.
- Permission enforcement.
- Migrations.
- Unit tests.

Exit criteria:

- Products and artefacts can be created and queried.
- Artefacts are workspace isolated.
- Schema validation works.
- Relationships are validated.
- Versions are immutable.

Phase 2: Basic UI

Build:

- Studio navigation.
- Studio Home.
- Product list.
- New Product wizard.
- Product overview.
- Artefact detail.
- Basic PRD projection.

Exit criteria:

- User can create a product through the UI.
- User can create and edit artefacts.
- PRD view renders from artefacts.

Phase 3: Canvas

Build:

- Graph API.
- Canvas page.
- Node cards.
- Edges.
- Filters.
- Node drawer.

- Relationship editing.
- Saved layouts.
- Auto-layout.

Exit criteria:

- User can visually explore a product.
- User can create and remove permitted relationships.
- Canvas performs acceptably with seed data.

Phase 4: AI Generation

Build:

- Studio specialist agents.
- Output schemas.
- Generation-run model.
- Stage orchestration.
- Background tasks.
- Realtime progress.
- Review checkpoints.
- Validation and repair.

Exit criteria:

- Idea can generate a connected product-definition draft.
- Invalid AI output is not stored.
- User can approve stages.

Phase 5: Reviews and Versioning

Build:

- Comments.
- Review requests.
- Approval flow.
- Version history.
- Diff UI.
- Specification publication.

Exit criteria:

- Approved content is version-specific.
- User can publish a PRD snapshot.

Phase 6: Quality

Build:

- Deterministic rules.
- Readiness scores.

- Findings UI.
- Governance agent.
- Resolution workflow.

Exit criteria:

- Missing traceability is visible.
- Blocking findings can prevent publication.

Phase 7: Engineering Handoff

Build:

- Architecture generation.
- API generation.
- Data generation.
- Test generation.
- Feature detail traceability.
- Coding-agent export.

Exit criteria:

- A feature can produce an engineering-ready context package.

Phase 8: GitHub Integration

Build:

- Product-repository link.
- Feature-issue link.
- Feature-pull-request link.
- GitHub status display.
- Approved issue creation.

Exit criteria:

- A feature is traceable to implementation work.

42. MVP Backlog

Epic STUDIO-001 Foundation

Tasks:

```
STUDIO-001-01 Add Studio feature configuration
STUDIO-001-02 Create Product model and migration
STUDIO-001-03 Create Artefact model and migration
STUDIO-001-04 Create ArtefactVersion model and migration
```

STUDIO-001-05 Create Relationship model and migration
STUDIO-001-06 Implement artefact schema registry
STUDIO-001-07 Implement relationship registry
STUDIO-001-08 Implement repositories and services
STUDIO-001-09 Implement workspace scoping
STUDIO-001-10 Add audit events

Epic STUDIO-002 Product APIs

STUDIO-002-01 Product CRUD
STUDIO-002-02 Artefact CRUD
STUDIO-002-03 Version APIs
STUDIO-002-04 Relationship APIs
STUDIO-002-05 Graph endpoint
STUDIO-002-06 Search and filtering
STUDIO-002-07 API error handling
STUDIO-002-08 API documentation

Epic STUDIO-003 Product UI

STUDIO-003-01 Studio navigation
STUDIO-003-02 Studio home
STUDIO-003-03 Product list
STUDIO-003-04 New Product wizard
STUDIO-003-05 Product overview
STUDIO-003-06 Artefact form framework
STUDIO-003-07 Artefact detail drawer
STUDIO-003-08 Empty and error states

Epic STUDIO-004 Canvas

STUDIO-004-01 Canvas route
STUDIO-004-02 Graph rendering
STUDIO-004-03 Node components
STUDIO-004-04 Edge components
STUDIO-004-05 Filters
STUDIO-004-06 Auto-layout
STUDIO-004-07 Saved positions
STUDIO-004-08 Relationship creation
STUDIO-004-09 Keyboard accessibility
STUDIO-004-10 Performance tests

Epic STUDIO-005 Generation

STUDIO-005-01 GenerationRun models
STUDIO-005-02 Generation plan API
STUDIO-005-03 Agent output contracts
STUDIO-005-04 Product Strategist Agent
STUDIO-005-05 Product Manager Agent
STUDIO-005-06 Architect Agent
STUDIO-005-07 QA Agent
STUDIO-005-08 Stage orchestration
STUDIO-005-09 Output validation
STUDIO-005-10 Realtime progress
STUDIO-005-11 Retry and cancellation
STUDIO-005-12 Generation UI

Epic STUDIO-006 Documents

STUDIO-006-01 Projection framework
STUDIO-006-02 PRD projection
STUDIO-006-03 Engineering handoff projection
STUDIO-006-04 Markdown export
STUDIO-006-05 Publication snapshots

Epic STUDIO-007 Collaboration

STUDIO-007-01 Comments
STUDIO-007-02 Mentions
STUDIO-007-03 Review requests
STUDIO-007-04 Approval decisions
STUDIO-007-05 Review Centre
STUDIO-007-06 Activity timeline

Epic STUDIO-008 Quality

STUDIO-008-01 Rule framework
STUDIO-008-02 Initial deterministic rules
STUDIO-008-03 Readiness calculation
STUDIO-008-04 Quality findings UI
STUDIO-008-05 Governance Agent
STUDIO-008-06 Publication blocking

Epic STUDIO-009 Coding-Agent Handoff

```
STUDIO-009-01 Feature context assembler  
STUDIO-009-02 Markdown export format  
STUDIO-009-03 JSON export format  
STUDIO-009-04 Claude Code profile  
STUDIO-009-05 Codex profile  
STUDIO-009-06 Export UI
```

Epic STUDIO-010 GitHub

```
STUDIO-010-01 Repository mapping  
STUDIO-010-02 Issue links  
STUDIO-010-03 Pull-request links  
STUDIO-010-04 GitHub status UI  
STUDIO-010-05 Human-approved issue creation
```

43. MVP Acceptance Criteria

The MVP is complete only when all of the following are true.

Product creation

1. An authorised user can create a product.
2. A product is isolated to its workspace.
3. Creating a product creates an initial idea artefact.
4. A user can select knowledge bases.
5. A user can select a product template.

Artefacts

1. At least the MVP artefact types are supported.
2. Artefact content is schema validated.
3. Artefacts can be created, read, updated and archived.
4. Every meaningful update creates an immutable version.
5. Version conflicts return a clear conflict response.

Relationships

1. Relationships are typed.
2. Invalid source and target combinations are rejected.
3. Cross-workspace relationships are impossible.
4. A product graph endpoint returns nodes and edges.

Generation

1. A user can generate a product definition from an idea.
2. Generation executes asynchronously.
3. Progress is visible in the UI.
4. Every stage stores status and output.
5. Invalid AI output is repaired once or rejected.
6. Generated content is saved as a draft.
7. AI metadata is recorded.
8. The user can approve or request changes to a stage.

Canvas

1. A user can open a visual product canvas.
2. The canvas supports zoom, pan and filters.
3. Selecting a node opens its details.
4. A user can create a valid relationship.
5. A user can navigate from an objective to linked implementation context.
6. Canvas layouts can be saved.

Documents

1. A PRD is rendered from graph artefacts.
2. Editing PRD content updates the related artefact.
3. A PRD can be exported to Markdown.
4. An approved PRD can be published as an immutable version.

Collaboration

1. Users can comment on artefacts.
2. Users can request reviews.
3. Reviewers can approve or request changes.
4. Approvals apply to exact versions.
5. Approved artefacts become draft again when edited.

Quality

1. The system detects missing required relationships.
2. The system shows readiness categories.
3. Blocking findings prevent publication when configured.
4. Every finding explains why it was created.

Engineering handoff

1. A feature detail page shows requirements, stories, criteria, architecture, APIs, data and tests.
2. A user can export feature context for Codex or Claude Code.
3. The export clearly distinguishes approved, draft and unresolved content.
4. The export contains no credentials.

Security and operations

1. All Studio endpoints enforce permissions.
 2. All material actions are audited.
 3. AI tools are scoped to configured permissions.
 4. Existing Synkora functionality continues to pass regression tests.
 5. Studio can be enabled through deployment configuration.
-

44. Definition of Done for Every Task

A task is complete only when:

1. The implementation follows existing repository conventions.
 2. Backend types and validation are present.
 3. Workspace and permission checks are present.
 4. Errors are handled using standard error responses.
 5. Audit events are emitted where required.
 6. Unit tests are added.
 7. Integration or component tests are added where appropriate.
 8. Documentation is updated.
 9. No unrelated functionality is changed.
 10. Linting, type checks and tests pass.
 11. Migration impact is documented.
 12. Security implications are reviewed.
 13. UI includes loading, empty and error states.
 14. Accessibility has been considered.
 15. No secrets or sensitive content are logged.
-

45. Instructions for Codex or Claude Code

Use the following execution rules.

45.1 Before coding

1. Read this complete specification.
2. Inspect `ARCHITECTURE.md`, repository setup documentation and contributor guidance.
3. Inspect existing coding conventions.
4. Inspect existing models, repositories, services and route patterns.
5. Inspect authentication and workspace isolation.
6. Inspect database migration tooling.
7. Inspect current agent, workflow, Celery and WebSocket implementations.
8. Inspect frontend component and API-client conventions.
9. Inspect testing and CI commands.
10. Create `docs/studio/repository-assessment.md`.
11. Create `docs/studio/implementation-plan.md`.
12. Do not start broad implementation until the repository assessment is complete.

45.2 Implementation behaviour

- Implement one phase at a time.
- Keep commits or change sets small.
- Do not refactor unrelated Synkora code.
- Prefer reuse over duplication.
- Introduce generic Core extension points only when necessary.
- Keep Studio-specific names and logic within Studio modules.
- Add tests with each phase.
- Run relevant validation commands after each phase.
- Fix failures before moving forward.
- Do not silently weaken permissions or validations.
- Do not add new infrastructure unless required.
- Do not introduce a graph database in the MVP.
- Do not create a separate Studio authentication system.
- Do not create a new AI runtime.
- Do not create direct provider calls outside Synkora's model layer.

45.3 When repository reality differs

When the specification's example paths differ from the actual repository:

1. Follow existing repository architecture.
2. Preserve the defined module boundaries.
3. Document the chosen path.
4. Do not change product behaviour without recording the deviation.
5. Add a section to `docs/studio/implementation-decisions.md`.

45.4 Required documentation

Create and maintain:

```
docs/studio/README.md
docs/studio/repository-assessment.md
docs/studio/implementation-plan.md
docs/studio/domain-model.md
docs/studio/api.md
docs/studio/agents.md
docs/studio/security.md
docs/studio/deployment.md
docs/studio/implementation-decisions.md
```

45.5 Stop conditions

Stop and document the issue rather than guessing when:

- Workspace isolation cannot be verified.
- Existing approval semantics are unclear.
- A schema migration could destroy existing data.

- A Core change would create circular dependencies.
- A requested integration requires credentials not available.
- An AI tool could perform external writes without approval.
- Existing architecture contradicts a critical security requirement.

For non-critical ambiguity, use the simplest implementation consistent with this specification and document the decision.

46. Recommended First Coding Prompt

Use this prompt for the first Codex or Claude Code session:

You are implementing Synkora Studio inside the existing Synkora repository.

Synkora Studio is a modular vertical application described in the attached specification. It must reuse Synkora's existing authentication, workspaces, agent runtime, model gateway, knowledge bases, workflows, background workers, WebSockets, integrations and observability.

Do not start feature implementation immediately.

First:

1. Read the full specification.
2. Inspect the repository architecture and contributor documentation.
3. Identify the exact backend, frontend, database, migration, background-task, agent, realtime, permission and testing conventions.
4. Map each proposed Studio module to the actual repository structure.
5. Identify reusable existing services.
6. Identify any required generic Core extension points.
7. Identify architectural or security risks.
8. Produce:
 - docs/studio/repository-assessment.md
 - docs/studio/implementation-plan.md
 - docs/studio/implementation-decisions.md

The implementation plan must divide work into independently testable phases. Do not modify production code in this first session unless a docs/studio directory must be created.

Important constraints:

- Studio may depend on Synkora Core.
- Synkora Core must not depend on Studio.
- Studio must be optional behind a feature flag.
- Existing Synkora behaviour must remain backward compatible.
- Use PostgreSQL for the Studio graph in the MVP.
- Never bypass workspace isolation.

- Never write invalid AI output to the database.
- AI changes must remain drafts until accepted or approved.
- External write actions require human approval.

47. Recommended Second Coding Prompt

After repository assessment:

Implement Phase 1 of Synkora Studio according to the approved implementation plan and the full Studio specification.

Phase 1 includes:

- Studio feature configuration
- Product model
- Artefact model
- ArtefactVersion model
- Relationship model
- Initial artefact schema registry
- Initial relationship registry
- Database migrations
- Repository and service layers
- Product CRUD API
- Artefact CRUD API
- Relationship API
- Workspace isolation
- Optimistic concurrency
- Audit integration
- Unit and integration tests

Do not implement the canvas or AI generation yet.

Requirements:

- Follow existing repository conventions exactly.
- Keep all Studio-specific code inside the designated Studio module.
- Reuse existing auth, workspace, error, audit and database utilities.
- Validate JSON content by artefact type and schema version.
- Create an immutable artefact version for every meaningful update.
- Reject invalid relationship type combinations.
- Return a conflict when expected versions differ.
- Add migrations and rollback support.
- Add API documentation.
- Run linting, type checking and relevant tests.
- Report changed files, design decisions, commands run and remaining issues.

48. Final Product Success Criteria

Synkora Studio succeeds when a Deriv team can:

1. Enter a product or feature idea.
2. Generate a structured first specification.
3. Review every generated artefact.
4. Explore the complete product visually.
5. Find missing requirements and traceability.
6. Approve a stable specification version.
7. Generate an engineering-ready context package.
8. Give that package to Codex or Claude Code.
9. Link implementation work back to the specification.
10. Keep the specification useful as the product evolves.

The final principle is:

Every product decision, requirement, design, implementation, test, deployment and outcome should remain connected.

That is the foundation of Synkora Studio and what makes it a true “Figma for Product Specs.”